Control Questions — Short Answers

Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis

李泽涛 / LiZetao | HDU 23320332 | Group 23320113 | Variant N/A | 19 October 2026

Q1. Why is bcrypt preferable to SHA-256 for passwords?

SHA-256 is fast, so a GPU can test billions of guesses per second, and unsalted digests are vulnerable to rainbow tables. bcrypt is deliberately slow, salts every password and has a tunable cost (12 in this project), which makes offline cracking far more expensive. Argon2id is the modern first choice; SHA-256 belongs to integrity checks and HMAC, not password storage.

Q2. What is the main difference between SAST and DAST?

SAST is white-box: it analyses source code without running the application, catches dangerous patterns early in CI and points to the exact line, but it cannot see runtime configuration and produces false positives. DAST is black-box: it tests the running application over HTTP or the UI, so it finds real runtime and access-control issues, but it runs later and may miss code paths. They complement each other; Bandit is the SAST tool here and OWASP ZAP is the DAST tool in Work 2.

Q3. How does a JWT work, what does it contain, and how is it verified?

A JWT has three parts: header.payload.signature. The header names the algorithm; the payload carries claims such as sub, iat, exp, iss, aud and jti; the signature is an HMAC or public-key signature over the first two parts. The server recomputes the signature, pins the expected algorithm, checks exp/iss/aud and only then trusts sub. The payload is signed, not encrypted, so it must not contain secrets; short TTLs or a jti denylist handle revocation.

Q4. What risks come from not auditing third-party dependencies?

Unaudited dependencies bring known CVEs, supply-chain attacks and abandoned packages into the product. One outdated template engine can become remote code execution, as EJS 3.1.6 did in this work, and transitive dependencies can change silently. The mitigation is to pin versions, run SCA on every build (pip-audit, npm audit), keep an SBOM and remove unused packages.

Q5. What are the advantages of Docker and Vulhub over installing vulnerable software directly?

Containers isolate the vulnerable service, pin its dependencies and can be deleted after the lab, so the host stays clean and the environment is reproducible. Vulhub provides ready-made compose files and exploitation notes. Direct installation risks infecting the host, breaking system packages and leaving artifacts behind. Docker is not perfect isolation, so containers should still be treated as untrusted.

Q6. What is the difference between a vulnerability and an exploit? What was reproduced?

A vulnerability is the weakness itself — EJS 3.1.6 concatenating outputFunctionName into generated JavaScript. An exploit is the concrete input that triggers it. We reproduced the exploit of CVE-2022-29078: a crafted query made the server execute id and write pwned.txt while still returning HTTP 200. The vulnerability exists regardless of attack; the exploit proves its impact.

Q7. Why is regression testing important after a fix?

A patch can break login, response formats or normal behaviour, and an incomplete fix can leave another attack path open. Regression tests show that the application still works and that the vulnerability is really gone. Here the fixed page still rendered, the fixed container had no pwned.txt, and all 21 API tests passed.

Q8. How does CVE reproduction help a pentester or SOC analyst?

Pentesters use CVE reproductions to build realistic proofs of concept, prioritise by exploitability and verify that a fix blocks the attack. SOC analysts use them to write detection rules, triage alerts, hunt for exploitation and prioritise patching, especially for CVEs in CISA KEV or with public exploits. Reproducing in an isolated lab also teaches safe, minimal-payload testing.